26128 / NGUYỄN VĂN HOÀNG / 20233413

Báo cáo cá nhân tuần 2

Đề tài: Asynchronous virtio-blk and I/O Schedulers. Học phần Hệ điều hành ET4290, học kỳ 2026.1. Kỳ báo cáo theo kế hoạch: 26/10-01/11/2026. Sprint Jira: 30/10-02/11/2026. Công việc và số liệu trong báo cáo được thực hiện, kiểm chứng ngày 01/11/2026 trên máy Hoàng; không suy diễn thành nhật ký làm việc cả tuần.

Tóm tắt tiến độ (cá nhân)

Trong kỳ này, phần việc của Nguyễn Văn Hoàng chuyển từ phân tích đường I/O và thiết kế bất đồng bộ ở tuần 1 sang triển khai một nguyên mẫu READ mức kernel. Repo được clone lại vào thư mục riêng; nhánh tích hợp kết hợp request pool và cơ chế wait của Hiếu với hàng đợi FCFS của Lợi. Phần bổ sung của Hoàng gồm đường try-submit không chờ, ánh xạ head tới request có generation, phân biệt completion stock/async và thu hồi descriptor theo đúng bên sở hữu. Kết quả và staging buffer vẫn tồn tại đến khi caller release request.

Nguyên mẫu được kiểm chứng bằng READ thật qua thiết bị virtio của QEMU trên cấu hình một và bốn CPU. Hai mươi lượt boot đạt đủ mười lăm nhóm kiểm tra mỗi lượt, bao gồm áp lực tài nguyên, tái sử dụng head, dữ liệu toàn block, xử lý lỗi có điều khiển và tiến triển khi không còn submit mới. Các test mock/timer kế thừa được chạy riêng, không dùng để khẳng định hành vi DMA. Bản production đã build lại và vượt sáu ca hồi quy của dự án. Sơ đồ draw.io, log, metadata và lệnh tái lập được lưu cùng đầu ra. Phần triển khai sẵn sàng để nhóm review; chưa push hay nghiệm thu thay reviewer. Phạm vi vẫn là READ một block, chưa thay buffer cache và chưa đánh giá tốc độ.

Task của Hoàng	Kết quả kỹ thuật
PTO-12 / W2-H01	Đã triển khai try-submit và kiểm chứng READ thật.
PTO-13 / W2-H02	Đã tách owner/reclaim; kiểm chứng mapping và drain.
PTO-19 / W2-G02	Đã tích hợp và chạy bộ kiểm thử CPUS=1/4; chờ review.

Tổng quan: 20/20 boot READ thật; 4/4 lượt selftest mock/timer; 6/6 ca hồi quy production. 15 nhóm ca là số kịch bản khác nhau; 20 lần boot là số lần lặp, không phải 300 test độc lập.

KẾT QUẢ THỰC HIỆN

Đầu vào, phạm vi và đóng góp

Đối chiếu đầu ra tuần 1

Tuần 1 đã có bản đồ virtio_disk_rw, descriptor lifecycle, interrupt path và sequence đề xuất. Kernel lúc đó chưa triển khai async; usertests/grind trên máy Hoàng chưa chạy. Tuần 2 sử dụng các phân tích đó và contract mới để hiện thực READ submit → queue → virtio → complete → wait/poll → release, đồng thời bổ sung hồi quy còn thiếu.

Phần việc	Nguồn / người thực hiện	Đóng góp trong bản tích hợp
Contract + pool	Hiếu: 9e9233f (gồm 5296c92, 1677f11)	Kế thừa pool, generation, wait/poll/release, selftest.
Queue + oracle	Lợi: 6526009 (gồm 7a0f195, 657e1b5)	Kế thừa FCFS, mock driver, oracle host.
Driver + integration	Hoàng, working tree trên b6b44bb	Try-submit, head owner/map, IRQ reclaim, nối API, kiểm thử thực.

Phạm vi đã triển khai

Kernel C API; chỉ ROOTDEV=1; READ một block BSIZE=1024; blockno < FSSIZE=2000; pool 16 slot, mỗi slot có staging buffer tĩnh; 8 descriptor dùng chung với stock; FCFS chọn theo enqueue_seq. Một khóa disk.vdisk_lock bảo vệ pool, queue, state, descriptor allocator và rings.

Giới hạn phải ghi khi báo cáo

Chưa tích hợp thay bread/bwrite; chưa giải quyết coherence giữa staging buffer và buffer cache khi có ghi đồng thời vào cùng block. Chưa có async WRITE, SCAN/LOOK, deadline, merge, cancellation/kill cleanup hay API async cho ứng dụng. Syscall io_selftest chỉ phục vụ test theo cờ build; production trả -1. Chưa benchmark latency, throughput hay chứng minh phần cứng xử lý song song.

Trạng thái review và Jira

PTO-12/13/19 được đọc trên Jira và đều đang In Progress, Description trống tại thời điểm kiểm tra. Báo cáo này ghi kết quả thực hiện và chuẩn bị nội dung cập nhật; không đăng comment, đổi trạng thái hay ghi reviewer đã duyệt. Hiếu cần review driver/ownership trước khi nhóm đánh dấu Done.

KẾT QUẢ THỰC HIỆN

PTO-12: Tách READ try-submit

Mục tiêu đạt được: virtio_disk_try_submit_read(req) chỉ nhận tài nguyên, dựng chain, publish và trả kết quả; không đợi thiết bị và không ngủ chờ descriptor. aio_read_submit() trả handle để caller quyết định poll hoặc wait sau đó.

Các bước đã thực hiện

1. Validate ở API trước khi cấp slot: out khác NULL, có process context, dev đúng ROOTDEV và blockno hợp lệ. Pool đầy trả IO_EAGAIN, không ghi đè output handle.

2. Cấp slot từ pool Hiếu, điền dev/blockno, tăng submitted, enqueue qua FCFS của Lợi và thử dispatch dưới cùng io_lock. Thiếu descriptor vẫn giữ request QUEUED và giữ thứ tự seq.

3. Dùng alloc3_desc() chung với stock. Nếu không đủ ba descriptor, rollback phần đã lấy, tăng eagain_desc và trả IO_EAGAIN; không publish chain chưa hoàn chỉnh.

4. Dựng header VIRTIO_BLK_T_IN, reserved=0, sector=(uint64)blockno*(BSIZE/512). Descriptor dữ liệu trỏ req->data dài 1024, có VRING_DESC_F_WRITE vì thiết bị ghi RAM. Descriptor cuối là status một byte, khởi tạo 0xff.

5. Ghi info[head]={req,gen,ASYNC}, đặt req->head và INFLIGHT trước publication. Ghi avail.ring, io_fence(), tăng avail.idx, io_fence(), notify queue 0; trả 0.

6. Giữ đường virtio_disk_rw() đồng bộ và cơ chế WRITE/log. Thêm owner STOCK vào đường cũ để cùng used ring vẫn phân biệt đúng bên thu hồi.

Minh chứng trực tiếp

T01 so mọi byte trong block với fixture host, không chỉ kiểm syscall trả 0. T07 giữ hai chain đã publish, gửi thêm ba request, quan sát inflight=2, queued=3 và free_desc=2; sau nhả hold, queue tự tiến triển. T10 giữ xử lý used để quan sát submit đã trả khi request còn INFLIGHT, poll=0; không gọi đó là số đo latency.

```
kernel/io_request.c:92    aio_read_submit
kernel/io_request.c:121   io_started_locked
kernel/virtio_disk.c:339  virtio_disk_try_submit_read
kernel/virtio_disk.c:383  INFLIGHT before avail publication
```

KẾT QUẢ THỰC HIỆN

PTO-13: Completion và reclaim

Đường ngắt đã triển khai

devintr() nhận IRQ virtio và gọi virtio_disk_intr(). Driver acquire disk.vdisk_lock, ACK thanh ghi MMIO, đặt fence và drain các entry used. Mỗi entry dùng used.id làm head; không suy luận request theo thứ tự submit hoặc theo blockno. PLIC complete vẫn do devintr() thực hiện ở vị trí cũ, không gọi thêm trong driver.

Trường hợp	Hành vi IRQ	Bên trả descriptor
STOCK	Giữ kiểm tra status stock; b->disk=0; wakeup(b).	Caller virtio_disk_rw sau khi thức dậy.
ASYNC	Kiểm owner, req, gen, head, INFLIGHT; đọc status; clear map; free chain; DONE/ERROR; wakeup(req).	Driver completion khi chain đã nằm trong used ring.

Vì sao không double-free hoặc mất kết quả

ASYNC không free descriptor tại aio_release(). IRQ đã trả chain sau DMA; release chỉ trả slot pool và tăng generation. STOCK vẫn free chain tại caller. Owner và req->head được xóa trước khi head được dispatcher sử dụng lại. Request DONE/ERROR giữ staging buffer riêng, nên kết quả của A không phụ thuộc header/status của head sau khi head đã chuyển sang B.

Tiến triển và wait/wakeup

Dispatcher được gọi sau enqueue, sau drain completion async và sau caller stock free_chain. Vì vậy request đang QUEUED có thể được gửi tiếp mà không cần producer tạo submit mới. Completion chuyển state/status trước wakeup; aio_wait của Hiếu đăng ký sleep_prepare(req) khi còn giữ khóa, nhả khóa mới sleep và dùng while kiểm lại. aio_data được siết để chỉ trả con trỏ ở DONE, trả NULL khi pending/error/stale.

Kiểm chứng và giới hạn

T08 giữ request A chưa release trong khi B tái dùng head; cả hai staging buffer vẫn đúng. T09 đảo thứ tự tiêu thụ hai used entry thật bằng hook kiểm thử; không tuyên bố QEMU tự hoàn tất ngược. ERROR ghi đè status 1/2 sau DMA thật để kiểm EIO, không coi là lỗi phần cứng thật. T11 đối chiếu driver/request counters và drain sau mười chu kỳ.

```
kernel/virtio_disk.c:395  reap_head_locked
kernel/virtio_disk.c:435  drain_used_locked
kernel/virtio_disk.c:461  virtio_disk_intr
kernel/io_request.c:357  aio_data
```

KẾT QUẢ THỰC HIỆN

PTO-19: Tích hợp và harness

Clone và ghép nhánh có nguồn gốc rõ ràng

Fresh clone lấy main e8fa3c1; nhánh week2/hoang-virtio được tạo từ nhánh pool/wait mới nhất của Hiếu. Merge origin/week2/loi và giải quyết xung đột Makefile/main.c để giữ đủ objects, init và selftests của cả hai bên. Local merge commit b6b44bb lưu lịch sử hai nhánh. Các thay đổi driver/test/tài liệu của Hoàng sau merge hiện chưa commit và chưa push.

Những chỗ phải sửa để ghép được

Khởi tạo FCFS từ io_init; nối aio_read_submit và callback io_started_locked; bổ sung include fs.h và spinlock.h thiếu trong queue/test để header dùng được; thêm object io_hw_test và cờ IO_HW_SELFTEST; tái dùng syscall selftest của Hiếu trong build kiểm thử. Thêm cumulative released và gauge done/error để kế toán sau nhiều lần tái sử dụng slot vẫn đúng. Cập nhật D10 trong contract, phân biệt counters tích lũy và số slot đang giữ.

Fixture và tính tái lập

scripts/check-week02-hoang.py tạo bản sao fs.img dùng tạm cho từng lượt. Host đọc superblock và bitmap, chỉ đặt pattern vào các block 1968-1999 sau khi xác nhận chúng chưa được cấp phát. Không sửa fixture vào fs.img gốc. Kernel test chỉ READ các block này; mỗi byte kỳ vọng là (block*17 + i*31 + (i>>8)*7) & 255. Mỗi lượt QEMU dùng ảnh mới.

Harness clean build trước mỗi lần đổi cờ để tránh make tái sử dụng object sai chế độ. Boot trực tiếp QEMU tránh make qemu tự relink mất cờ. Timeout, panic, thiếu marker, sai thứ tự FCFS, sai dữ liệu hoặc kế toán đều làm FAIL. Sau các selftest, clean build production và dùng nm xác nhận test-hook symbols không còn.

Phép kiểm âm và hồ sơ thực nghiệm

Một ảnh tạm bị đảo đúng một byte fixture để oracle phải báo sai pattern. Kết quả EXPECTED_FAIL được ghi riêng, không cộng vào PASS READ. meta.json lưu thời điểm, phiên bản công cụ, HEAD dirty, hash source và hash kernel/fs theo chế độ; source.patch kèm bản sao source mới chưa tracked để có thể tái dựng. Lượt debug nhanh trước bộ chính chỉ dùng để phát hiện lỗi, không cộng thêm vào 20 lượt nghiệm thu.

KẾT QUẢ THỰC HIỆN

Ma trận 15 nhóm kiểm tra

Tất cả các nhóm dưới đây PASS trên mỗi lượt của 10 boot CPUS=1 và 10 boot CPUS=4. Thiết bị QEMU xử lý READ thật; các điều kiện có hook được nêu rõ. Ký hiệu Txx nối với contract tuần 2; ARGS và ERROR là nhóm bổ sung.

Nhóm	Điều kiện / tiêu chí quan sát
T01	READ một block, so đủ 1024 byte pattern độc lập từ host.
ARGS	NULL out / dev sai / block vượt biên; output handle giữ nguyên.
T02	16 slot được giữ; submit thứ 17 EAGAIN; sau wait/release drain.
T03	Cùng slot đổi gen; handle cũ bị từ chối ở poll/wait/data/release.
T04	Release INFLIGHT và QUEUED trả EBUSY; data pending NULL.
T05	IRQ hoàn tất trước wait; wait fast path trả đúng kết quả.
T06	Reap used entry DMA thật trong gap sau sleep_prepare bằng hook.
T07	Hold reap: hai inflight, ba queued, hai free desc; unhold tự dispatch.
T08	Head được B sử dụng lại khi A vẫn DONE; buffer A/B không lẫn.
T09	Đảo tiêu thụ một cặp used thật bằng hook; mapping vẫn đúng.
ERROR	Gán status 1/2 sau DMA thật: poll/wait EIO, data NULL, release được.
T10	Submit đã trả khi còn INFLIGHT, poll=0, chưa terminal.
T12	READ stock và async cùng bản kernel; kiểm stock counter, data và drain.
T13	Block offset 7,2,9,1,4; host đòi dispatch seq 1,2,3,4,5.
T11	Mười chu kỳ bốn READ; đủ slot/desc sau từng chu kỳ, counters khớp.

T06/T09/T10 dùng điều khiển test-only để tạo tình huống khó tái lập; không thay thế mọi lịch chạy đồng thời có thể có. T12 của harness là ca trộn API; T12 trong kế hoạch rộng hơn còn được bao phủ bằng hồi quy production. Chưa có stress nhiều producer async đồng thời hay bằng chứng IRQ thật hoàn tất đúng tại khe prepare/sleep trên mọi lịch.

KẾT QUẢ THỰC HIỆN

Kết quả đo và hồi quy production

Số liệu từ 20 lượt READ thật

Chỉ số cuối mỗi lượt	Giá trị / diễn giải
submitted / released / completed_async	83 / 83 / 83
completed / errors	81 / 2; hai ERROR có tiêm status test-only.
free_req / free_desc	16 / 8; queued=inflight=done=error=0.
max_inflight	2 request đã publish và chưa được phần mềm reap.
eagain_desc	18-26; có thử thiếu descriptor và rollback.
head_reuse	42-43; reuse khi slot cũ còn terminal.

Không xem max_inflight=2 là benchmark. Với NUM=8 và mỗi chain dùng 3 descriptor, phần mềm giữ được tối đa hai chain cùng lúc; hook hold reap giúp quan sát con số này, kể cả khi DMA đã xong nhưng chưa được xử lý.

Hồi quy trên bản production đã bỏ cờ test

Case	CPU	Kết quả	Giây
boot-smoke	1	PASS	1.45
shell-fs-cpus3	3	PASS	1.45
usertests-quick-cpus1	1	PASS	85.60
usertests-quick-cpus4	4	PASS	75.19
grind-smoke-cpus1	1	PASS	60.79
grind-smoke-cpus4	4	PASS	60.79

Hai ca usertests -q có marker ALL TESTS PASSED. grind là kiểm sống trong cửa sổ 60 giây không panic theo oracle có sẵn; không phải chứng minh ổn định vô hạn. Bộ cũ mang tag baseline và note stock; lần chạy này dùng kernel đã sửa, nên được báo là hồi quy, không phải baseline stock mới.

Chạy riêng test pool/wait mock + timer CPU1/4: PASS 2/2; mock FCFS với oracle host của Lợi CPU1/4: PASS 2/2. Build bốn chế độ và kiểm production hooks: PASS. git diff --check và make check-text đã qua. Phiên hồi quy đầu bị ngắt trước khi lưu ca grind cuối; lưu riêng hồ sơ interrupted và dùng regression-final hoàn chỉnh làm kết quả chính.

KẾT QUẢ THỰC HIỆN

Bất biến, đồng bộ và review

Kế toán đúng sau nhiều lần release

```
S = Q + I + C + E
C + E = R + D + X
16 = F + Q + I + D + X
Sau drain: S = C + E = R; F=16; free_desc=8
```

S=submitted, C=completed, E=errors, R=released là cumulative counters. Q=queued, I=inflight, D=done, X=error, F=free_req là số slot hiện tại. Snapshot giữ io_lock; ALLOCATED chỉ tồn tại nội bộ trước khi submit nhả khóa. Các selftest cấp slot/chuyển state trực tiếp của Hiếu không dùng phương trình cumulative của luồng API thật.

Các điểm cần người review xác nhận

1. Không có sleep/kalloc/copyout/buf unlock trong try-submit hoặc IRQ; alloc3_desc rollback đầy đủ trước EAGAIN. 2. Map req/gen/owner và INFLIGHT trước avail.idx/notify. 3. Mọi used.id qua kiểm tra head/owner/state; stock caller và async IRQ không cùng free một chain. 4. Dữ liệu chỉ đưa ra ở DONE, slot giữ đến release; stale handle không alias khi gen đổi. 5. Cả ba trigger dispatcher có mặt và giữ đúng một lock domain.

Những điều chưa thể kết luận

Không có bằng chứng tốc độ cải thiện; chưa đo latency/throughput hoặc disk utilization. Chưa chạy usertests đầy đủ không -q, stress 30 phút, nhiều producer async, wrap generation 32-bit, kill/cancel hoặc lỗi thiết bị thật. CPUS=4 kiểm sự vận hành trên bốn hart và cạnh tranh với IRQ/stock; không đồng nghĩa test nhiều tiến trình cùng submit async.

Việc chuyển tiếp đề nghị

Hiếu review ownership và state/counters trên diff; Lợi đối chiếu trace FCFS khi tích hợp; cả nhóm chốt tag/API/READ-only với GV qua PTO-7 đang có. Sau review, thêm bài stress nhiều producer và quy tắc cancel/kill, rồi mới tích hợp cache hoặc thuật toán tuần sau. Không thay đổi phạm vi sang async WRITE trước khi có thiết kế thứ tự log/write.

KẾT QUẢ THỰC HIỆN

Chạy lại và xem bằng chứng

Làm việc tại repo clone mới, nhánh week2/hoang-virtio. Repo cũ trong HDH được giữ nguyên. Các lệnh build/test tạo lại fs.img trong clone kiểm thử; không dùng image đang chứa dữ liệu cần giữ.

```
cd /home/ubuntu/2026.1/HDH/
cd Asynchronous-virtio-blk-and-IO-Schedulers-week02-hoang-2026-11-01
git branch --show-current
git rev-parse HEAD
make test-week02
# Equivalent command (default repeat=10)
python3 scripts/check-week02-hoang.py --repeat 10
# Harness restores the production build before returning
python3 tests/runner.py --tag baseline
make check-text
git diff --check
make qemu CPUS=1
# Exit QEMU: Ctrl-A, then X
```

Mong đợi từ harness

```
IO-HWTEST: T13 seq=1,2,3,4,5
IO-HWTEST: stats submitted=83 completed=81 errors=2
  released=83 max_inflight=2 free_req=16 free_desc=8
IO-HWTEST: PASS failures=0
IO-USER-TEST: PASS (rc=0)
```

Lệnh kiểm nhanh: python3 scripts/check-week02-hoang.py --repeat 1 --hw-only. Lệnh này vẫn build và restore production, nhưng bỏ test mock/timer độc lập. Nếu đổi cờ thủ công, luôn make clean trước; make không theo dõi thay đổi CFLAGS để tự rebuild toàn bộ.

Hồ sơ chính trong repo

```
results/week02/hoang-20261101-213746/
  meta.json; summary.md; source.patch
  untracked-source/; build-*.log
  hw-cpus1-01.log ... hw-cpus4-10.log
  negative-corrupt-fixture.log
  pool-cpus*.log; queue-cpus*.log
  regression-final/meta.json; summary.md; logs/
```

meta.json ghi source SHA-256 và HEAD dirty b6b44bb; report generator so lại source với hash trước khi xuất báo cáo. Snapshot bàn giao gồm bundle lịch sử Git, patch các file tracked và source mới để không phụ thuộc nhánh remote sau này.

KẾT QUẢ THỰC HIỆN

Danh mục thay đổi và sơ đồ

File / nhóm file	Phần Hoàng thêm hoặc sửa so với merge b6b44bb
kernel/virtio_disk.c	Owner STOCK/ASYNC; try-submit READ; head→req/gen; IRQ drain/reclaim; dispatch trigger; stats và hook có cờ.
kernel/io_request.c / .h	Nối submit vào pool/FCFS; io_started_locked; released + done/error; aio_data chỉ DONE; real-used gap hook.
kernel/io_queue.c / io_queue_test.c	Include fs.h/spinlock.h để code của Lợi build cùng header. Thuật toán FCFS được kế thừa.
kernel/io_request_test.c; user/init.c	Nối IO_HW_SELFTEST vào entry selftest; giữ production syscall stub.
kernel/io_hw_test.c (mới)	15 nhóm test READ thật, pattern, mapping, resource pressure và drain.
scripts/check-week02-hoang.py (mới)	Build tách chế độ, ảnh tạm, oracle host, lặp CPU1/4, negative control, hashes/logs, restore production.
Makefile; mk/project.mk	Object/cờ build và target make test-week02.
docs/week02/interface.md	Sửa tên Lợi; D10 cumulative accounting; bổ sung API/status thống nhất.
docs/week02/hoang.md; diagrams/	Báo cáo source và draw.io ba trang có thể chỉnh sửa.

Cách mở sơ đồ

Dùng diagrams.net: File → Open From → Device → hoang-submit-complete.drawio. Chọn ba tab ở đáy: 01 Async submit-complete; 02 Ownership va hai vong doi; 03 Kiem thu va minh chung. Mọi hình và mũi tên là đối tượng draw.io, không phải ảnh phẳng. Chưa ghi đè bản sơ đồ web cũ của nhóm; bản mới mô tả code tuần 2 đã triển khai.

Phụ lục hình phía sau được xuất từ đúng file draw.io này. Trang sequence được tách phần submit và completion để dễ đọc trong PDF; khi chỉnh sửa, mở file draw.io nguyên bản.

KẾT QUẢ THỰC HIỆN

Nội dung cập nhật Jira và trình bày

PTO-12 - kết quả để điền Description / báo cáo

Đã triển khai virtio_disk_try_submit_read: cấp 3 descriptor có rollback EAGAIN; dựng READ 1024 byte; map req/gen và INFLIGHT trước avail/notify; không sleep. aio_read_submit trả handle, giữ request QUEUED khi thiếu desc. Bằng chứng: T01/T07/T10 PASS trên 10 boot CPU1 và 10 boot CPU4. Xem virtio_disk.c:339 và log hw-cpus*. Chờ Hiếu review trước Done.

PTO-13 - kết quả để điền Description / báo cáo

Đã xử lý completion theo used.id và owner STOCK/ASYNC; stock caller free, async IRQ free; clear map trước head reuse, giữ slot/data đến release. Wakeup sau terminal state; dispatch tiếp sau reclaim. T05/06/08/09/11 và nhánh EIO đều PASS CPU1/4. Đảo used/status lỗi là test hook được ghi rõ. Xem virtio_disk.c:395 và docs/week02/hoang.md; chờ peer review.

PTO-19 - kết quả để điền Description / báo cáo

Đã tích hợp pool/wait Hiếu và FCFS Lợi trên week2/hoang-virtio; merge base b6b44bb, source Hoàng đang dirty. Harness READ thật PASS 20/20 boot × 15 nhóm ca; 4/4 lượt mock/timer; production regression PASS 6/6 gồm usertests -q và grind 60s CPU1/4. Sau drain 16 slot/8 desc; test hooks vắng trong production. Có draw.io, PDF, hashes và log. Chưa push; đề nghị review rồi commit/PR.

Kịch bản báo cáo 8-10 phút

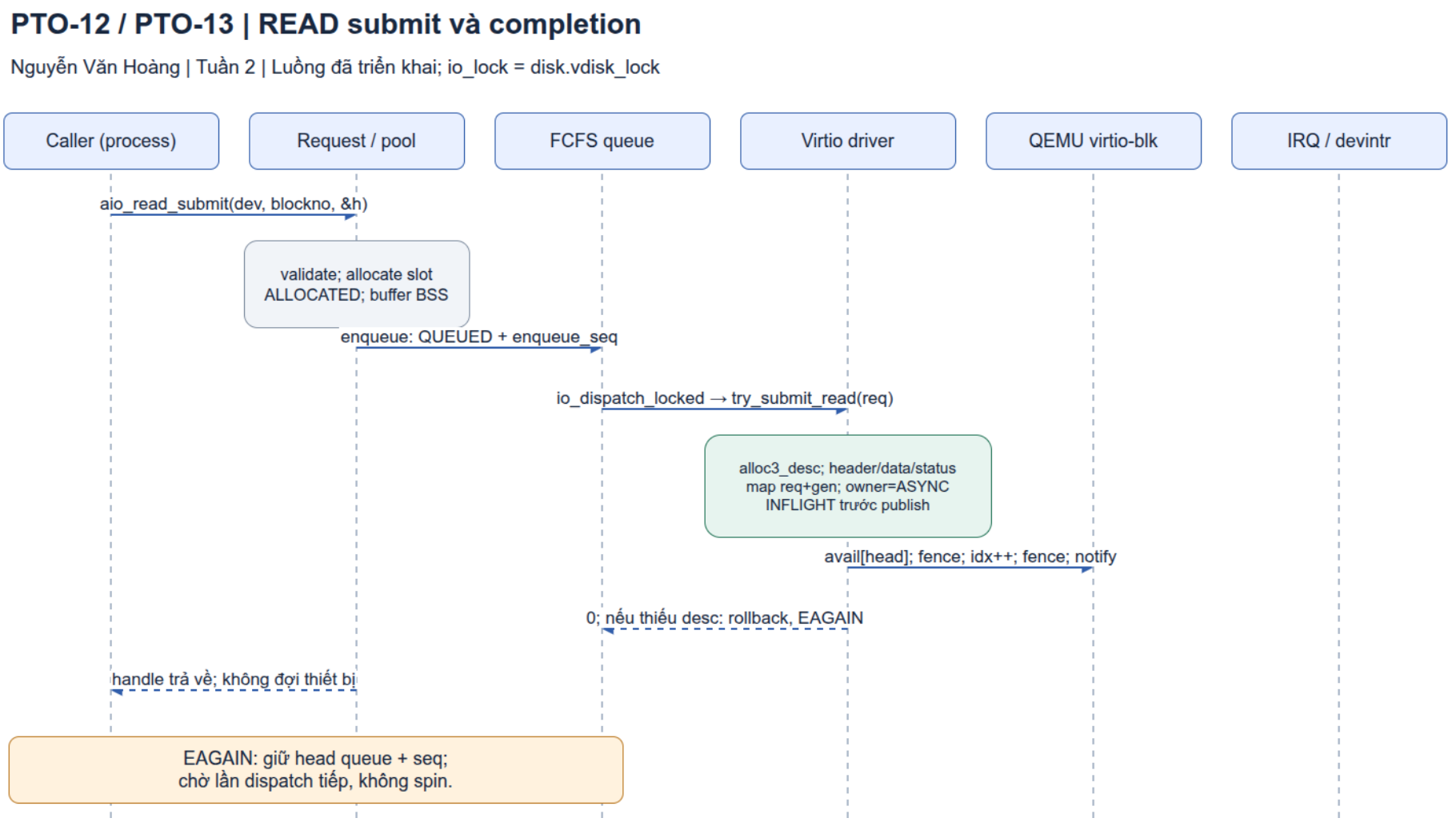
Phút 0-1: nêu ba task, phạm vi READ một block và việc kế thừa hai đồng đội. Phút 1-3: mở sequence, chỉ map/state trước publication và submit trả trước wait. Phút 3-5: mở owner/lifecycle, giải thích ai free chain và vì sao A còn đọc dữ liệu khi head đã chuyển sang B. Phút 5-7: mở log T07/08/09/11, các counters 83/81/2/83 và free 16/8. Phút 7-9: mở regression-final/summary.md, nêu usertests -q và grind 60s. Phút cuối: nêu review pending và các giới hạn.

Câu hỏi cần tự giải thích: Vì sao READ có bit WRITE? Vì device ghi RAM. Vì sao NUM=8 chỉ giữ hai chain? Vì mỗi READ cần ba descriptor. Vì sao DONE chưa FREE? Caller còn cần buffer. max_inflight=2 có chứng minh disk chạy song song? Không. Vì sao cần cả MMIO ACK và PLIC complete? Hai tầng báo nhận khác nhau.

Nguồn đối chiếu: docs/week02/interface.md; kernel/io_request.h; docs/week01/hoang.md; Doc/Spec/Progress Report template.docx. Repo: github.com/HieuPahm-R2/Asynchronous-virtio-blk-and-IO-Schedulers. Jira: phamvanthang786.atlassian.net/browse/PTO-12 (tương tự PTO-13, PTO-19). Đây là báo cáo chi tiết cá nhân, dùng làm phụ lục cho báo cáo tiến độ của cả nhóm. Không ghi giờ công khi chưa có nhật ký xác nhận.

SƠ ĐỒ XUẤT TỪ DRAW.IO

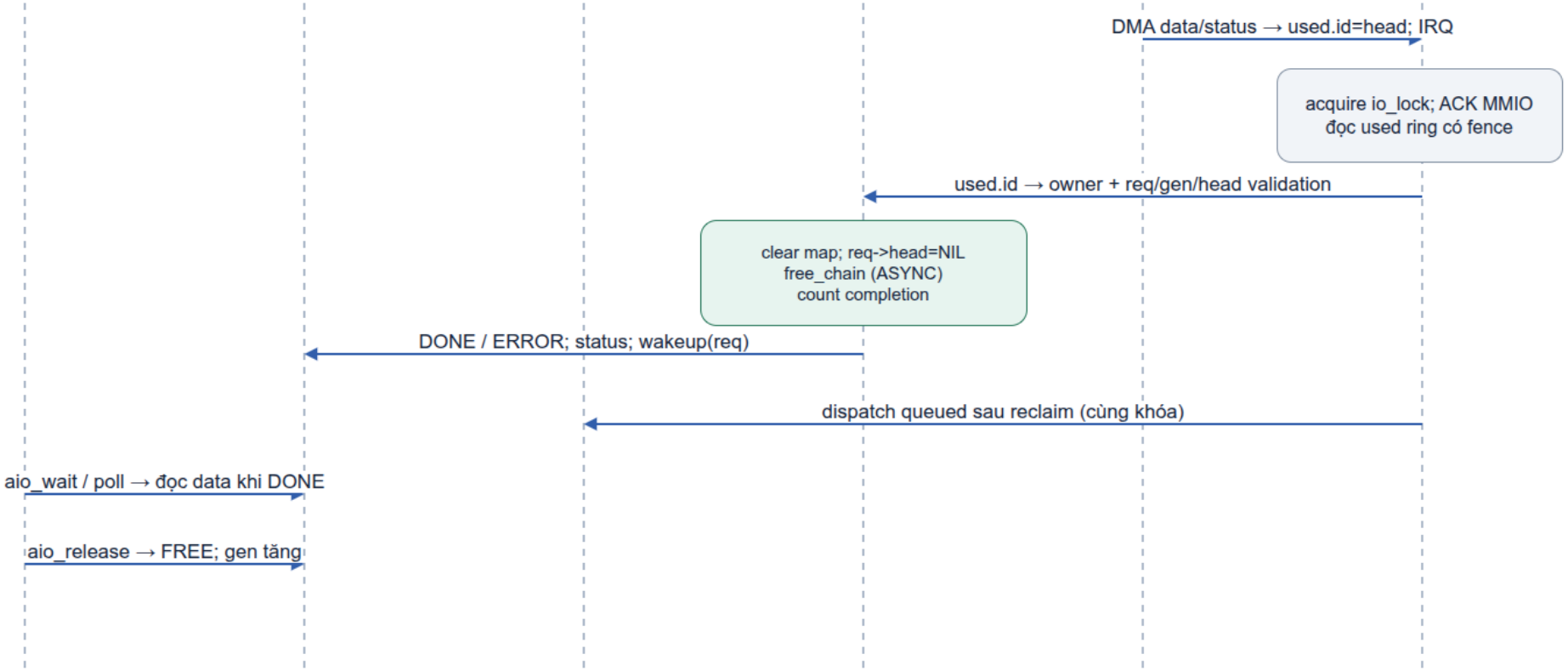
Phụ lục A | Sequence: submit



Phần đầu của trang draw.io 01. Các thao tác pool/queue/driver chạy dưới cùng io_lock; đường try-submit không ngủ.

SƠ ĐỒ XUẤT TỪ DRAW.IO

Phụ lục B | Sequence: completion



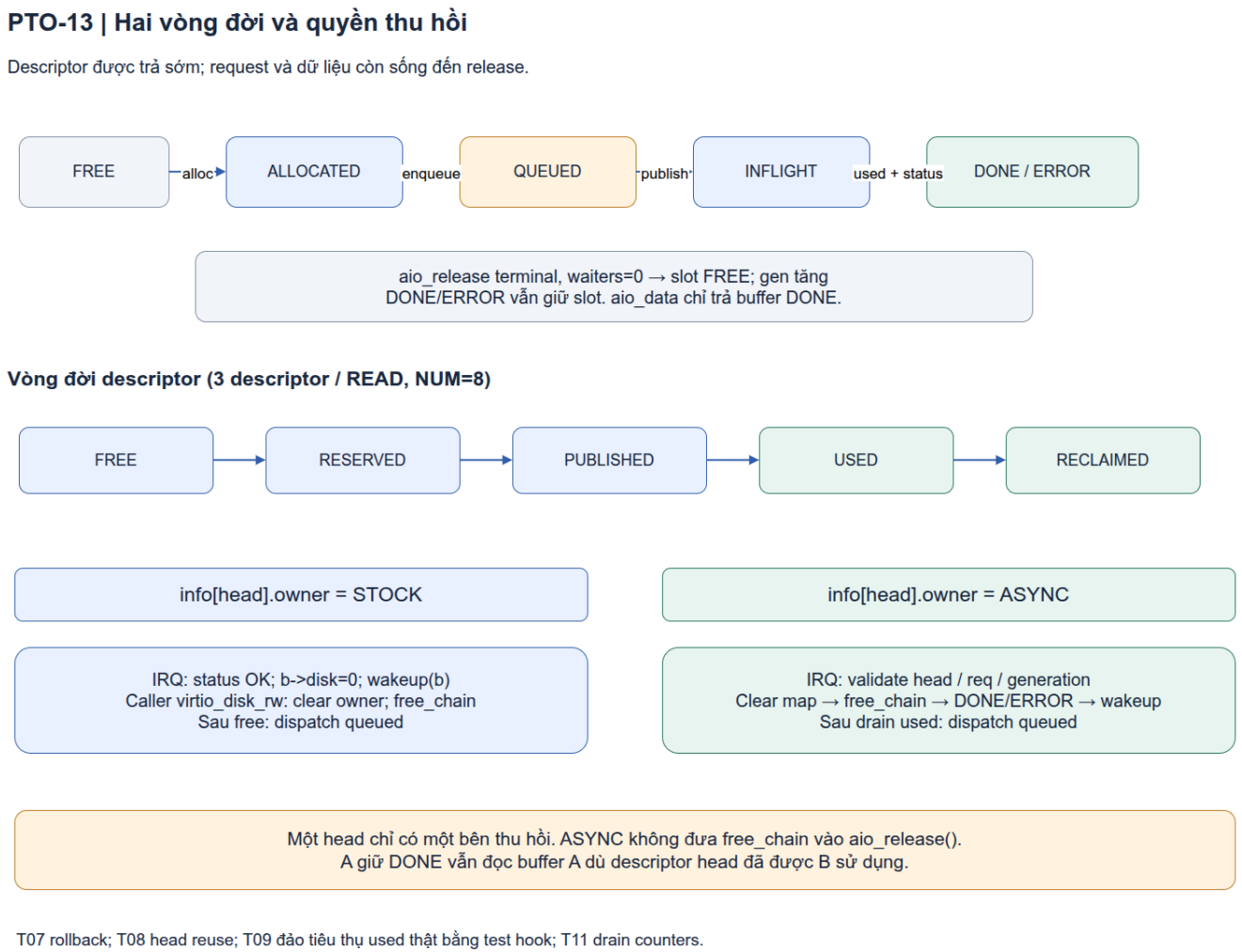
PLIC complete do devintr() thực hiện sau driver; giữ vị trí cũ. IRQ không free request slot.

Ghi chú: nếu caller wait trước completion: sleep_prepare(req) dưới io_lock → unlock → sleep() → lock → kiểm lại while.

Lanes từ trái sang phải: Caller; Request/pool; FCFS; Driver; QEMU; IRQ. Callback terminal và wakeup chạy sau khi DMA đặt head vào used.

SƠ ĐỒ XUẤT TỪ DRAW.IO

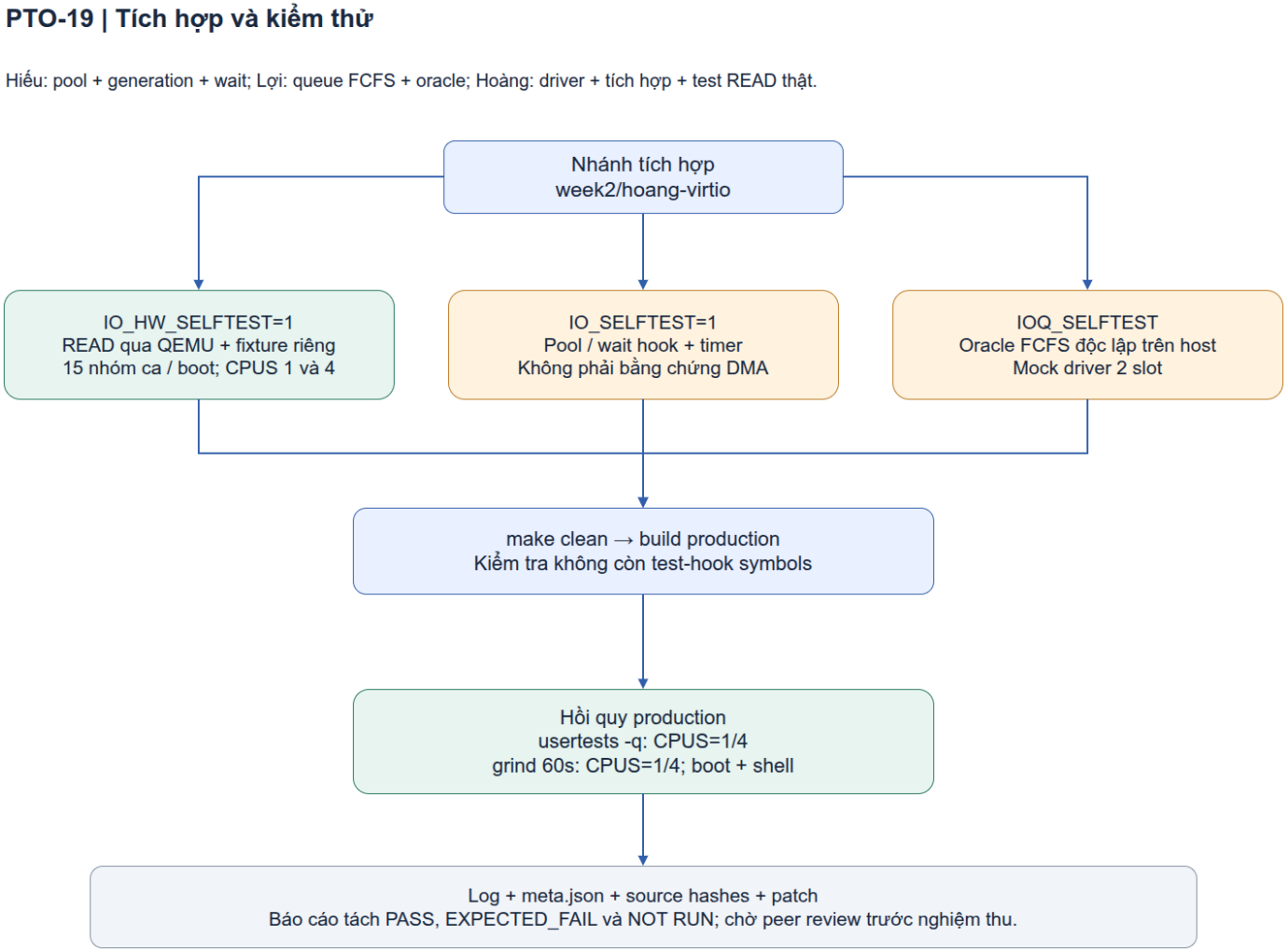
Phụ lục C | Ownership và vòng đời



Trang draw.io 02. Cùng một used ring, hai quy tắc thu hồi descriptor; slot và kết quả có vòng đời dài hơn chain.

SƠ ĐỒ XUẤT TỪ DRAW.IO

Phụ lục D | Tích hợp và kiểm thử



Trang draw.io 03. Luồng READ thật, mock/timer và hồi quy production có bằng chứng riêng, không gộp thành số đo hiệu năng.